

Apertura: martes, 3 de diciembre de 2024, 08:30

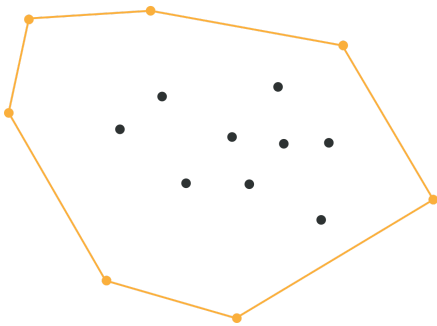
Cierre: jueves, 19 de diciembre de 2024, 16:30

Objetivo

El objetivo principal de esta práctica consiste en **desarrollar un ejemplo de algoritmo basado en la estrategia general algorítmica Divide y Vencerás (*Divide & Conquer*) para obtener la envoltura convexa de un conjunto de puntos en el plano**. Para ello se detalla a continuación una introducción al problema, los requisitos funcionales mínimos, ejemplos de uso y los criterios de evaluación.

Introducción

[Quickhull](#) es un algoritmo diseñado para calcular la [envoltura convexa](#) de un conjunto finito de puntos, específicamente en el contexto bidimensional (2D). Dicho algoritmo implementa una estrategia basada en el principio de *divide y vencerás*, análoga a la empleada por el algoritmo de ordenación [Quicksort](#), de donde deriva su nomenclatura.

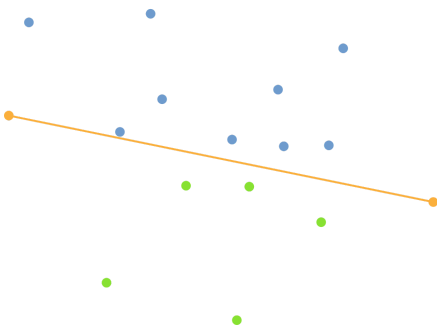


En términos de complejidad algorítmica, Quickhull exhibe una complejidad promedio de $\Theta(n \log(n))$. Sin embargo, en escenarios de alta simetría o cuando los puntos se distribuyen formando una circunferencia, la complejidad puede degenerar a orden cuadrático en el peor caso.

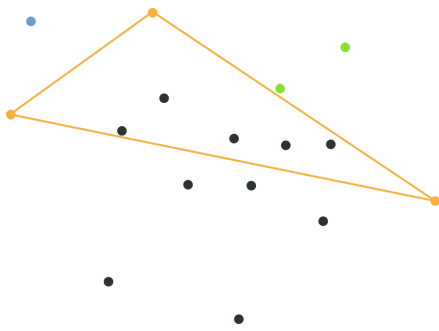
Algoritmo Quickhull

El [algoritmo Quickhull](#) puede desglosarse en los siguientes pasos:

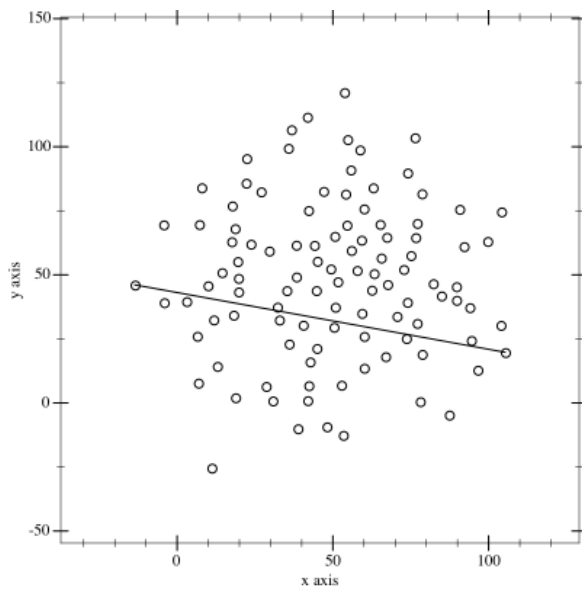
- Identificar los puntos con las coordenadas X mínima y máxima, dado que estos necesariamente pertenecerán al cierre convexo.



- Utilizar el segmento que conecta ambos puntos para particionar el conjunto de puntos en dos subconjuntos, cada uno de los cuales serán procesados de manera recursiva.
- Determinar el punto que se encuentra a la mayor distancia de la línea formada por los dos puntos identificados inicialmente. Este punto, junto con los dos previamente seleccionados, conformará un triángulo.



- Descartar todos los puntos situados en el interior del triángulo, ya que no serán parte de la envoltura convexa.
- Repetir los dos pasos anteriores para ambos lados del triángulo, excluyendo el lado inicial.
- Continuar el proceso hasta que todos los puntos hayan sido clasificados. Los puntos seleccionados constituirán la envoltura convexa.



La clase *point_set*

```

namespace CyA
{
    typedef std::pair<double,double> point;
    typedef std::pair<point, point> line;
    typedef std::vector<point> point_vector;

    enum side
    {
        LEFT = -1,
        CENTER,
        RIGHT
    };

    class point_set : public point_vector
    {
    private:
        point_vector hull_;

    public:
        point_set(const vector<point> &points);
        ~point_set(void);

        void quickHull(void);

        void write_hull(std::ostream &os) const;
        void write(std::ostream &os) const;

        inline const point_vector& get_hull(void) const { return hull_; }
        inline const point_vector& get_points(void) const { return *this; }

    private:
        void quickHull(const line &l, int side);

        double distance(const line &l, const point &p) const;
        int find_side(const line &l, const point &p) const;
        void x_bounds(point &min_x, point &max_x) const;
        double point_2_line(const line &l, const point &p) const;
        bool farthest_point(const line &l, int side, point &farthest) const;
    };
}

```

```

void point_set::quickHull(void)
{
    hull_.clear();

    CyA::point min_x_point;
    CyA::point max_x_point;

    x_bounds(min_x_point, max_x_point);

    quickHull(CyA::line(min_x_point, max_x_point), side::LEFT);
    quickHull(CyA::line(min_x_point, max_x_point), side::RIGHT);

    // Remove duplicates
    std::sort(hull_.begin(), hull_.end());
    hull_.erase(std::unique(hull_.begin(), hull_.end()), hull_.end());
}

```

```

void point_set::quickHull(const CyA::line &l, int side)
{
    CyA::point farthest;

    if (farthest_point(l, side, farthest))
    {
        quickHull(CyA::line(l.first, farthest), -find_side(CyA::line(l.first, farthest), l.second));
        quickHull(CyA::line(farthest, l.second), -find_side(CyA::line(farthest, l.second), l.first));
    }
    else
    {
        hull_.push_back(l.first);
        hull_.push_back(l.second);
    }
}

```

```
bool point_set::farthest_point(const CyA::line &l, int side, CyA::point &farthest) const
{
    farthest = CyA::point_vector::at(0);

    double max_dist = 0;

    bool found = false;

    for (const CyA::point &p : *this)
    {
        const double dist = distance(l, p);

        if (find_side(l, p) == side && dist > max_dist)
        {
            farthest = p;
            max_dist = dist;
            found = true;
        }
    }

    return found;
}
```

```
double point_set::point_2_line(const CyA::line &l, const CyA::point &p) const
{
    const CyA::point &p1 = l.first;
    const CyA::point &p2 = l.second;

    return (p.second - p1.second) * (p2.first - p1.first) -
           (p2.second - p1.second) * (p.first - p1.first);
}

double point_set::distance(const CyA::line &l, const CyA::point &p) const
{
    return fabs(point_2_line(l, p));
}
```

Requisitos funcionales mínimos

Para superar la práctica, los requisitos funcionales mínimos exigidos son:

- Implementar el algoritmo Divide y Vencerás Quickhull para obtener la envoltura convexa de un conjunto de puntos utilizando como referencia, y no obligatoriamente, el esqueleto proporcionado en este enunciado
- La clase debe adaptarse al estilo exigido en las prácticas, [Google C++ Style Guide](#)
- El programa realizado debe ejecutarse leyendo desde la entrada estándar el conjunto de puntos, y emitiendo su resultado a la salida estándar

Requisitos opcionales

Como requisito opcional, se pide mostrar la salida utilizando, al igual que en la práctica anterior, un fichero DOT para ser visualizado por GRAPHVIZ.

Ejemplos

Se pueden encontrar algunos ejemplos para probar la solución en [esta carpeta](#).

Evaluación

Se evaluará positivamente los siguientes aspectos:

- Funcionamiento correcto de la modificación solicitada: si no se consigue realizar la modificación la práctica será valorada con un cero.
- Presentación en el laboratorio: el grado de funcionamiento de la práctica, y si desarrolla mejoras significativas.
- Código subido en la tarea correspondiente a la práctica: buen diseño, estructura de clases y limpieza.

Teniendo superados los aspectos anteriormente descritos, una evaluación orientativa sería la siguiente:

- Si se desarrollan los requisitos funcionales mínimos: hasta 7 puntos.
- Si se desarrolla además una mejora significativa junto a los requisitos opcionales: hasta 10 puntos.

Profesor responsable

Si surgiera alguna **duda más sobre la práctica, el enunciado o la corrección**, podría formularse a través del [correo electrónico del profesor Jorge Riera](#) (grupo de mañana) o del [correo electrónico del profesor José Luis González Ávila](#) (grupo de tarde).